Stata for Survey Microdata

A Practical Introduction

Alexia Sánchez-Vázquez

Contents

1 Getting Started with Stata **3**

1.1 What is Stata? . . . 3

1.2 The Stata interface . . . 3

1.3 Working with do-files . . . 4

1.3.1 Comments . . . 4

1.3.2 A first do-file . . . 5

1.4 Anatomy of a command . . . 5

1.5 Loading and looking at data . . . 6

1.6 Microdata and household surveys . . . 7

1.6.1 What are microdata? . . . 7

1.6.2 The EFF and the HFCS . . . 7

2 Creating Variables, Missing Values and Loops **9**

2.1 Creating and changing variables: `generate` and `replace` . . . 9

2.2 Missing values . . . 10

2.3 Extended generate: `egen` . . . 11

2.3.1 Row functions: across variables, within one row . . . 11

2.3.2 Group functions: across rows, within a group . . . 11

2.4 Repeating commands: macros and loops . . . 12

2.4.1 Local macros . . . 12

2.4.2 `foreach` and `forvalues` . . . 12

2.4.3 Using results inside a loop . . . 12

3 Working with Confidential Survey Microdata **14**

3.1 Rules of the game . . . 14

3.2 The mock dataset . . . 14

3.3 Stacking the implicates: `append` . . . 15
3.4 Labels and dummies: `label` and `tabulate, generate()` . . . 16
3.5 Weighted statistics: `summarize, detail`, `quietly` and `scalar` . . . 16
3.6 A quick table by implicate: `table` . . . 17
3.7 Results as data: `preserve`, `collapse` and `restore` . . . 17
3.8 Standard errors: `expand` and the `mi` commands . . . 18
3.9 Taking results out: `export excel` . . . 19
3.10 Summary . . . 20
References . . . 20

Chapter 1

Getting Started with Stata

These notes are written for economists who know what a weighted mean, a median or a standard error is, but have never used Stata, and who need to work with household survey microdata such as the Spanish Survey of Household Finances (EFF) or the euro area Household Finance and Consumption Survey (HFCS). No previous programming experience is assumed: every command is explained piece by piece.

By the end of this chapter you should be able to open Stata, find your way around its windows, read the structure of any command, write a short do-file, and load and inspect a dataset.

1.1 What is Stata?

Stata is a program for statistical analysis. You give it a dataset and a list of instructions (*commands*), and it returns tables, statistics, graphs and estimates. It is widely used in economics and is particularly well suited to survey data.

A dataset in Stata is a table:

- each **row** is an *observation* (for example, one household);
- each **column** is a *variable* (for example, household income).

Stata keeps *one* dataset in memory at a time, and every command acts on that dataset. Nothing you do in memory changes the file on your disk until you explicitly save it.

1.2 The Stata interface

When you open Stata you see five windows. Their position differs slightly between Windows and Mac, but their role is always the same:

Window	What it does
Command	Where you type a command and press Enter to run it.
Results	Where Stata prints the output of every command, and its error messages.
History	A list of the commands you have run in this session.
Variables	The variables in the dataset currently in memory.
Properties	Details of the selected variable (type, label, format).

A sixth window is the one you will use most: the **Do-file Editor**, a text editor where you write commands and save them in a file. Open it from the toolbar, or by typing `doedit` in the Command window.

When a command fails, Stata prints a message in red in the Results window, followed by a code such as `r(111)`. The message usually says what went wrong; clicking on the code opens a longer explanation.

1.3 Working with do-files

There are two ways to give Stata instructions:

1. **Interactively**: typing commands one by one in the Command window (or clicking through menus).
2. **With a do-file**: writing all commands in a text file with extension `.do` and running the file.

Key idea

Always work with do-files. Stata has *no undo button*: if you delete or change data by mistake, the only way back is to reload the original data and run your commands again. A do-file is a written record of every step, so anyone (including you, six months from now) can reproduce your results exactly.

In the Do-file Editor you can run the whole file or only the lines you have selected (the *Execute* button, or Ctrl+D on Windows and Cmd+Shift+D on Mac). Running a selection is convenient for testing, but it has side effects that Chapters 2 and 3 point out.

The files you will use most are:

Extension	Content
`.dta`	A Stata dataset.
`.do`	A do-file: plain text with commands.
`.log`	A log: a text record of the commands run and their output.

1.3.1 Comments

Comments are notes for humans; Stata ignores them.

```
* A line starting with an asterisk is a comment
clear all          // text after two slashes is a comment
/* Everything between these
   two markers is a comment */
summarize price ///
    weight         // three slashes: the command continues on the next line
```

Common error

The markers `//`, `///` and `/* */` only work inside a do-file. Typed in the Command window, they produce an error. If you copy a command split with `///` into the Command window, first join it into a single line and delete the slashes.

Good practice

Comment generously, and explain *why* rather than *what*: "`// non-owners have no home value: set to zero`" is more useful than "`// replace with zero`". Write comments and variable names in one language, preferably English, so that the code can be shared.

1.3.2 A first do-file

Most do-files start the same way:

```
* ------------------------------------------------------------
* Project : practice
* File    : 01_first_look.do
* Purpose : first look at a dataset
* Author  : your name, date
* ------------------------------------------------------------
version 19                      // run with the rules of this Stata version
clear all                       // empty the memory
capture log close               // close any log left open
cd "/path/to/my/project"        // set the working folder
log using first_session, replace text   // start recording

* ... your commands go here ...

log close                       // stop recording
```

- `version` tells Stata which version's rules to follow, so that the do-file gives the same results if you open it with a newer Stata. Write the version you use.
- `cd` (*change directory*) tells Stata which folder to work in. `pwd` shows the current folder.
- `capture` runs the command that follows and, if it fails, carries on instead of stopping the do-file. Here it avoids an error when no log is open.
- `replace` allows Stata to overwrite a file that already exists.

Common error

A path with spaces must be in double quotes: `cd "/My Documents/project"`. Without quotes, Stata reads only up to the first space and reports that the folder does not exist.

Good practice

Give every project the same folder structure, for example `data/` (original files, never modified), `do/` (do-files), `output/` (tables, graphs, logs). Number do-files in the order they must run (`01_build.do`, `02_analysis.do`) and keep the header block at the top of each one.

1.4 Anatomy of a command

Almost every Stata command follows the same pattern. Learning it once explains most error messages.

```
command varlist if in [weight] , options
```

Part	Meaning
`command`	What to do: `summarize`, `tabulate`, ...
`varlist`	The variables it acts on: `income age`.
`if`	Which observations: `if region == "N"`.
`in`	Which rows by position: `in 1/10`.
`[weight]`	Survey weights, in square brackets: `[aw = weight]`.
`,`	The comma separates the command from its options.
`options`	How to do it: `detail`, `replace`, `by(group)`.

For example, in `summarize income if region == "N" [aw = weight], detail` the command is `summarize`, it acts on `income`, only for households in region N, weighted by `weight`, and the option `detail` asks for percentiles as well as the mean.

Not every command accepts every part. The help file of each command (`help summarize`) starts with its exact syntax: parts in square brackets are optional, and anything not listed is not allowed.

Common error

Typical messages when a command is used outside its syntax:

- `if not allowed r(101)`: the command does not accept an `if` (for example `describe`).
- `too many variables specified r(103)`: `tabulate` accepts at most two variables.
- `option ... not allowed r(198)`: the option does not exist for that command.

To see the values of some variables for some households, the command is `list`: `list hh_id income if hh_id == 3`. `display` is not for variables: it shows a single number, text or calculation, such as `display 2 + 2`.

1.5 Loading and looking at data

Stata ships with small example datasets, which are useful for practising. The classic one describes 74 car models from 1978:

```
sysuse auto, clear      // load a built-in example dataset; clear = drop what is in
    memory
```

To open your own file use `use`: `use "mydata.dta", clear`.

Common error

- `no; dataset in memory has changed since last saved r(4)`: you tried to open a dataset while another one, modified, is in memory. Add the option `clear`.
- `file mydata.dta already exists r(602)`: `save` refuses to overwrite a file. Add the option `replace`, but *never* on an original data file.

Each of these commands answers a different question about the data:

Command	Question it answers
`describe`	How many observations and variables are there, and what are they?
`browse`	What do the data look like? (spreadsheet view; read only)
`list in 1/5`	What are the values in the first five rows?
`codebook price`	What type of variable is this, and how many values are missing?
`summarize price`	What are its mean, standard deviation, minimum and maximum?
`tabulate foreign`	How many observations fall in each category?
`count`	How many observations are there (that satisfy a condition)?

Common error

A single `=` assigns a value; a double `==` compares two values. `summarize price if foreign = 1` gives `invalid syntax r(198)`; the correct condition is `if foreign == 1`.

Good practice

In do-files, write commands and variable names in full rather than abbreviated (`summarize`, not `su`). Stata also accepts abbreviated *variable* names, which can silently pick the wrong variable; `set varabbrev off` at the top of a do-file switches this off.

1.6 Microdata and household surveys

1.6.1 What are microdata?

Microdata are data on individual units (people, households, firms), one row per unit, rather than aggregated figures such as a national average. In a household survey, a sample of households answers a questionnaire, and each household becomes one or more rows in the dataset.

Three features of survey microdata matter from day one.

1. Weights. Some types of household are deliberately interviewed more often than others (wealth surveys oversample wealthy households). Each household therefore comes with a *weight*: the number of households in the population it represents. Statistics computed without weights describe the sample, not the population.

2. Missing values. Some questions do not apply to some households (a renter has no home value) and some respondents do not answer. In Stata a missing value is written as a dot, `.`.

Be careful

For Stata, a missing value is *larger than any number*. A condition such as `if income > 1000` therefore also selects households whose income is missing. Write `if income > 1000 & !missing(income)` instead. Stata gives no error here: the result is simply wrong.

3. The unit of observation. Before any analysis, ask: what is a row? A household, a person, or something else? Surveys often come in several files with different units (one for households, one for persons), linked by an identifier.

1.6.2 The EFF and the HFCS

The EFF, run by the Banco de España, and the HFCS, run by the Eurosystem and coordinated by the European Central Bank, collect detailed information on household wealth, debt, income

and consumption. Their microdata are available to researchers on request, under confidentiality rules. Compared with a simple survey, they have two extra features that Chapter 3 works through:

- **Multiple imputation.** When a household does not answer a question, the missing value is filled in (*imputed*) five times, so each household appears five times in the data, once per *implicate*.
- **Replicate weights.** In the HFCS, besides its main weight, each household has 1,000 additional weights, used to compute correct standard errors.

Good practice

Never modify the original data files. Read from them, and save your own work in a separate folder. Note in your do-file header which file and which version of the data you used (for example, the wave and the release date).

Chapter 2

Creating Variables, Missing Values and Loops

This chapter shows how to *build* new variables from existing ones, how to handle missing values correctly, and how to repeat a command many times without copying it. These are the tools that turn raw survey answers into the variables of an analysis, such as total household income.

Throughout the chapter we use a small invented dataset with the structure of household survey microdata. Paste it into a do-file and run it:

```
clear                                   // empty the memory
input hh_id person_id weight inc_wage inc_pension
1 1 1500 20000     .
1 2 1500     .  9000
2 1 2300     .     .
3 1  800 35000     0
3 2  800 12000     .
end                                     // end of the data
```

Each row is a *person*. Persons are grouped into households (`hh_id`); `weight` is the household weight; `inc_wage` and `inc_pension` are two income components; a dot is a missing value. `input` lists the variable names, then one line per observation, and `end` closes the list.

2.1 Creating and changing variables: `generate` and `replace`

`generate` (short form `gen`) creates a *new* variable. `replace` changes the values of a variable that *already exists*.

```
* New variable: wage income in thousands of euros
generate inc_k = inc_wage / 1000

* Dummy: 1 if wage above 30,000, 0 otherwise, missing if wage unknown
generate high_wage = (inc_wage > 30000) if !missing(inc_wage)

* Change an existing variable: round to whole thousands
replace inc_k = round(inc_k)
```

- An expression in parentheses, such as `(inc_wage > 30000)`, equals 1 if it is true and 0 if it is false. This is the quickest way to create a dummy.
- The `if` at the end of the second command leaves the dummy missing when the wage is missing. Without it, those persons would get a 1, because a missing value counts as larger than any number.
- `replace` has no short form: Stata makes you type it in full, because it overwrites data.

Common error

- `variable inc_k already defined r(110)`: you used `generate` for a variable that exists. Use `replace`, or `drop` it first.
- `variable inc_k not found r(111)`: you used `replace` (or any command) with a variable that does not exist. Check the spelling; Stata distinguishes upper and lower case.

Be careful

With `generate`, arithmetic involving a missing value gives a missing value: `generate total = inc_wage + inc_pension` is missing for every person with at least one missing component. Section 2.3 shows the alternative.

Good practice

Do not overwrite original survey variables. Create a new variable with a clear name and leave the original untouched, so that you can always compare the two. Give every new variable a label that says what it is and in which units: `label variable inc_k "Wage income, thousand EUR"`.

2.2 Missing values

Three tools cover most situations:

Tool	What it does
`missing(x)`	Equals 1 if `x` is missing and 0 otherwise. Works for numbers and text.
`count if missing(x)`	Counts the missing values of `x`.
`mvdecode x, mv(-1 -2)`	Converts numeric codes for "don't know" or "not applicable" (here -1 and -2) into missing values.

A missing value can mean two different things, and you must decide which one before computing anything:

- **Not applicable.** The question was not asked because it does not apply: a household that does not own a home has no home value. For many purposes the right value is *zero*.
- **Not known.** The question applied but there is no answer. The right value is *missing*, and it must not become a zero.

Be careful

Some surveys code "don't know" or "not applicable" as negative numbers (such as -1 or -2) or as very large ones (such as 999999). If you do not convert them into missing values first, every mean you compute is wrong, and Stata will not warn you.

Good practice

Before replacing missing values with zeros, find the variable that says whether the item exists ("owns a home: yes/no") and use it in the condition: `replace home_value = 0 if missing(home_value) & owns_home == 0`. The survey documentation states, for each

question, to whom it is asked (its *filter*). Read it before deciding.

2.3 Extended generate: egen

`egen` creates variables with functions that `generate` does not have. There are two families.

2.3.1 Row functions: across variables, within one row

```
* Sum of the components; missing components count as zero
egen inc_person = rowtotal(inc_wage inc_pension)

* Same, but the total is missing if ALL components are missing
egen inc_person2 = rowtotal(inc_wage inc_pension), missing

* Number of missing components in each row
egen n_missing = rowmiss(inc_wage inc_pension)
```

Be careful

`rowtotal` treats missing values as zeros. A person whose components are *all* missing (person 1 of household 2) gets a total of 0, which reads as "no income" although it means "unknown". The option `missing` keeps the total missing in that case. Compare both versions with `list inc_wage inc_pension inc_person inc_person2`.

2.3.2 Group functions: across rows, within a group

Combined with `bysort`, `egen` computes a statistic for each group and writes it on every row of the group. This is how person-level information becomes household-level information:

```
* Household income: sum over the members of each household
bysort hh_id: egen inc_hh = total(inc_person2), missing

* Household size: number of members
bysort hh_id: egen n_members = count(person_id)
```

`bysort hh_id:` sorts the data by household and runs the command separately for each household. Other useful functions are `mean()`, `max()`, `min()` and `tag()`; see `help egen`.

Once the household variables exist, keep one row per household and check it:

```
* _n is the row number within each household: keep the first row only
bysort hh_id: keep if _n == 1

* Stop with an error if hh_id does not identify each row
isid hh_id
```

Good practice

Check every step. `isid` confirms that a set of variables identifies each row; `assert` stops the do-file if a condition is false (`assert n_members >= 1`); `count if` tells you how many

rows meet a condition. A do-file that stops with an error is better than one that silently produces wrong numbers.

2.4 Repeating commands: macros and loops

2.4.1 Local macros

A *local macro* is a name that stores a piece of text. You define it with `local` and use it by writing its name between a backtick (`) and an apostrophe (').

```
local components inc_wage inc_pension    // store the text "inc_wage inc_pension"
summarize `components'                   // Stata reads: summarize inc_wage
    inc_pension
```

Common error

A local only exists while the selection that defines it is running. If you run the `local` line, and then run the `summarize` line separately, the macro is empty and Stata reads `summarize` alone: it summarises *every* variable, with no error message. Always run the definition and its use in the same selection.

2.4.2 `foreach` and `forvalues`

A loop runs the same commands several times, changing one element each time.

```
* Over variables: v takes the value inc_wage, then inc_pension
foreach v of varlist inc_wage inc_pension {
    count if missing(`v')
}

* Over numbers: k takes the values 1, 2, 3
forvalues k = 1/3 {
    display "This is round `k'"
}
```

Common error

The opening brace `{` must be the last thing on the line of `foreach` or `forvalues`, and the closing brace `}` must be on a line of its own. Writing a command on the same line as a brace stops the loop with an error.

Good practice

Indent the commands inside a loop by four spaces, as in the examples. Stata does not need it, but nested loops become unreadable without it.

2.4.3 Using results inside a loop

After most commands, Stata stores results that you can reuse. After `summarize`, `r(mean)` holds the mean and `r(N)` the number of observations; type `return list` to see them all. These results are overwritten by the next command that stores results.

```
* meanonly: compute the mean without printing a table
summarize inc_hh [aw = weight], meanonly
display "Weighted mean household income: " r(mean)
```

Key idea

Why loops matter for the EFF and the HFCS. Each household appears five times, once per *implicate*. Most calculations are therefore done five times and then combined, which is a natural job for `forvalues m = 1/5`. Group functions must also respect this structure: with implicates, a household is identified by its identifier *and* the implicate (and, in the HFCS, the country), so aggregation uses `bysort country hh_id implicate:` rather than `bysort hh_id:`.

Chapter 3

Working with Confidential Survey Microdata

The EFF and the HFCS are distributed under strict confidentiality rules. You work on a protected computer, the original files must stay where they are, and only aggregated results may leave. This chapter introduces the twelve commands you will use most often in that setting.

Instead of real data, we build a small *mock dataset* with the same structure as the files a data provider sends. Run every block on your own computer as you read.

3.1 Rules of the game

Every data provider has its own contract, but four rules are almost universal.

1. **Read, never write, the original files.** Load them with `use` and save your own work somewhere else.
2. **Only aggregated results leave the protected environment**: means, medians, shares, regression coefficients. Never individual records.
3. **Keep intermediate microdata in memory**, not in new files. `preserve` and `restore` (Section 3.7) are built for this.
4. **Check how many observations lie behind each number.** A mean computed on three households can reveal information about them. Providers often set a minimum number of observations per published figure.

Be careful

A log file records everything printed in the Results window. If you run `list` while a log is open, individual records end up in the log. Treat a log as microdata unless it only contains aggregated output.

Good practice

Define the paths once, at the top of a master do-file, with *global macros*, and use them everywhere else: `global rawdata "/secure/eff/2022"` and then `use "$rawdata/file.dta", clear`. If the data move, you change one line. Unlike locals, globals survive between selections until Stata is closed.

3.2 The mock dataset

Our invented survey has eight households in two regions (`N` and `S`). For each household we know its age group, its weight, its annual income, whether it owns its home and, if so, the home's value. Three values were not reported: the income of households 3 and 6, and the home value of household 7.

Like the EFF and the HFCS, the provider fills in (*imputes*) these values five times and delivers *five files*, one per implicate. The code below creates them. Run it once, in an empty practice folder:

```
clear
* str1 = region is text, one character long
input hh_id str1 region age_group weight income owns_house house_value
1 "N" 1 1200 18000 0      .
2 "N" 2  900 42000 1 210000
3 "N" 3 1500     . 1 150000
4 "S" 1 2000 15000 0      .
5 "S" 2 1100 39000 1 180000
6 "S" 3  700     . 1 320000
7 "S" 2 1300 27000 1      .
8 "N" 3  600 55000 1 400000
end

* Simulate five imputed files (the data provider would do this for you)
forvalues m = 1/5 {
    preserve                                          // keep a copy of the data
    replace income      = 30000 + 1000*`m'  if hh_id == 3
    replace income      = 22000 + 1500*`m'  if hh_id == 6
    replace house_value = 200000 + 5000*`m' if hh_id == 7
    save toy_imp`m', replace                          // toy_imp1.dta ... toy_imp5.
    dta
    restore                                           // back to the copy
}
```

The five files are identical except for the three imputed values. Note that `house_value` is still missing for households 1 and 4: they do not own a home, so the value is *not applicable*, not unknown.

Common error

Text values must be in double quotes when you compare them: `list if region == "N"`. Without quotes (`region == N`) Stata looks for a *variable* called `N` and reports `N not found r(111)`. Comparing a text variable with a number (`region == 1`) gives `type mismatch r(109)`.

3.3 Stacking the implicates: append

`append` adds the rows of another file below the data in memory. We use it to stack the five implicates into one dataset, with a variable that records which implicate each row comes from.

```
use toy_imp1, clear                    // implicate 1 in memory
generate implicate = 1                 // mark its rows
forvalues m = 2/5 {
    append using toy_imp`m'            // add the rows of implicate m below
    replace implicate = `m' if missing(implicate)   // mark the new rows
}
isid hh_id implicate                   // one row per household and implicate
tabulate implicate                     // 8 rows in each implicate
```

Each appended file arrives without the variable `implicate`, so it is missing in the new rows; the `replace` line fills it in.

Key idea

After stacking, each household appears five times. A household is identified by `hh_id` *and* `implicate` together, which is what `isid` checks. With several countries, the country code joins the identifier as well.

Try it. Run `list hh_id implicate income if hh_id == 3`. Which values change across implicates, and which do not?

3.4 Labels and dummies: `label` and `tabulate, generate()`

Survey variables are coded as numbers. Value labels attach a text to each code, so that output is readable.

```
* Step 1: create a list of texts, named age_lbl
label define age_lbl 1 "Under 40" 2 "40-64" 3 "65 and over"
* Step 2: attach the list to the variable
label values age_group age_lbl
tabulate age_group if implicate == 1
```

The codes do not change, only how they are displayed. Conditions still use the codes: `if age_group == 3`, not `if age_group == "65 and over"`.

`tabulate` with the option `generate()` creates one 0/1 variable per category. The mean of a 0/1 variable is a share, so these dummies give "the share of households in each group" directly.

```
tabulate age_group, generate(age_)   // creates age_1, age_2, age_3
```

3.5 Weighted statistics: `summarize, detail`, `quietly` and `scalar`

With multiply imputed data, any statistic is computed in each implicate separately and then averaged over the five:

$$\bar{\theta} = \frac{1}{5}\sum_{m=1}^{5}\hat{\theta}_m.$$

Three tools do this in a loop:

- `summarize, detail` computes the mean and the percentiles, and stores them in `r()`: `r(mean)` is the mean and `r(p50)` the median. Type `return list` after it to see everything it stores.
- `quietly` runs a command without printing its output. The command still runs and still stores its results.
- `scalar` stores one number under a name. Because `r()` is overwritten by the next command, a scalar is how you keep a result: it stays until you close Stata or delete it with `scalar drop`.

```
forvalues m = 1/5 {
    * Compute, without printing, the weighted statistics of implicate m
    quietly summarize income [aw = weight] if implicate == `m', detail
    * Copy the results into named scalars before they are overwritten
```

```
    scalar mean_`m' = r(mean)
    scalar p50_`m'  = r(p50)
}
* Average of the five implicates
display "Mean income:   " (mean_1 + mean_2 + mean_3 + mean_4 + mean_5) / 5
display "Median income: " (p50_1 + p50_2 + p50_3 + p50_4 + p50_5) / 5
```

Why `aw` and not `pw`? Survey weights are, strictly, *probability weights* (`pw`): the inverse of the probability that the household was sampled. For a point estimate (a mean, a median) analytic weights (`aw`) give exactly the same number. They differ only in how standard errors are computed, and `summarize` does not report standard errors. The rule: `aw` for descriptive commands that only give the number; `pw` whenever you need standard errors (Section 3.8).

Common error

`pweights not allowed r(101)`: `summarize` does not accept `pw`. Use `aw` for the point estimate, or the `svy` commands of Section 3.8 for standard errors.

Be careful

Do not compute a statistic on the five implicates pooled together and call it the result. For a mean this happens to give the same number, but for a median it generally does not, and the standard errors are wrong in both cases.

3.6 A quick table by implicate: `table`

`table` shows several statistics for each group in one command. It is a fast way to see how much the implicates differ.

```
table implicate [aw = weight], statistic(mean income) ///
    statistic(median income) nototals
```

Piece by piece: `table implicate` gives one row per implicate; `[aw = weight]` weights each household; each `statistic()` adds a column (here the mean and the median of `income`); `nototals` removes the "Total" row, which would pool the five implicates. This syntax requires Stata 17 or later, and the table only displays results: it does not store them.

3.7 Results as data: `preserve`, `collapse` and `restore`

`collapse` replaces the data in memory with a much smaller dataset of statistics: one row per group. Because it destroys the microdata in memory, it always goes between `preserve` (take a snapshot of the data) and `restore` (return to the snapshot).

```
preserve                                    // snapshot of the 40 rows
* Step 1: one row per implicate, with its mean and median
collapse (mean) mean_inc = income (median) p50_inc = income ///
    [aw = weight], by(implicate)
list                                        // 5 rows
* Step 2: average of the five implicates
collapse (mean) mean_inc p50_inc
list                                        // 1 row: the result
restore                                     // back to the 40 rows
```

In `(mean) mean_inc = income`, the statistic is in parentheses, the name of the new variable on the left and the original variable on the right. The same two steps give results by group: add the group to `by()` in step 1 and keep it in step 2.

```
preserve
collapse (mean) mean_inc = income [aw = weight], by(implicate age_group)
collapse (mean) mean_inc, by(age_group)
list
restore
```

Common error

`preserve` only lasts while the selection that contains it is running: when the selection ends, Stata restores the data automatically. If you run `preserve` and the following lines as separate selections, your changes disappear between them, and a later `restore` answers that there is nothing to restore. Run each block from `preserve` to `restore` in a single selection.

Good practice

`preserve` and `restore` are the main reason you rarely need to save intermediate microdata to disk. If you do need a temporary file, a `tempfile` is deleted automatically when the do-file ends; check that your provider allows it.

3.8 Standard errors: `expand` and the `mi` commands

The averages above give the right point estimates, but not standard errors. Stata's `mi` commands combine the five implicates with Rubin's rules and report correct standard errors. Three steps are needed.

1. Add an "implicate 0". `mi` expects, alongside implicates 1 to 5, the original data with the missing values, labelled implicate 0. Providers do not deliver it, but `mi` does not use it in the calculations, so a copy of implicate 1 will do. `expand` repeats rows: `expand 2` turns each selected row into two, the original and a copy, and places the copies at the end of the data. The option `generate()` creates a marker variable equal to 0 for the original rows and 1 for the copies.

```
preserve
expand 2 if implicate == 1, generate(copy)   // duplicate implicate 1; copy == 1
    marks the new rows
replace implicate = 0 if copy == 1           // the copies become implicate 0
drop copy                                    // the marker is no longer needed
tabulate implicate                           // 8 rows in each of 0, 1, ..., 5
```

The code book of the Banco de España for the EFF does the same in two more lines: it creates a variable `aux` equal to 1, sets it to 2 for implicate 1, and runs `expand aux`, which repeats each row as many times as `aux` says. The result is identical.

2. Declare the structure. `mi import flong` tells Stata which variable identifies the household (`id()`) and which identifies the implicate (`m()`). `mi svyset` declares the survey weight, here as a probability weight because we want standard errors.

```
mi import flong, id(hh_id) m(implicate) clear
mi svyset [pweight = weight]
```

3. Estimate. Put `mi estimate:` in front of a survey command.

```
mi estimate: svy: mean income                      // mean with standard error
mi estimate: svy: mean income, over(age_group)     // by group
mi estimate: svy: proportion owns_house            // shares
restore                                            // back to the stacked data
```

In the output, `Coefficient` is the point estimate (it should equal the result of `collapse`) and `Std. err.` its standard error.

Common error

Steps 1 to 3 must run in a single selection, from `preserve` to `restore`. Otherwise implicate 0 disappears when the first selection ends (see Section 3.7), and `mi import` fails. If the number of observations used differs across implicates, `mi estimate` also stops with an error; the option `esampvaryok` (`mi estimate, esampvaryok: ...`) allows it, once you have checked that the difference is expected.

Key idea

In the HFCS, standard errors also require the 1,000 replicate weights. They are declared in `mi svyset` with the options `bsrweight()` and `vce(bootstrap)`. A later chapter covers this step.

3.9 Taking results out: `export excel`

Once a dataset contains only aggregated results, it can be exported.

```
preserve
collapse (mean) mean_inc = income [aw = weight], by(implicate age_group)
collapse (mean) mean_inc, by(age_group)
export excel using "results_income.xlsx", firstrow(variables) replace
restore
```

`firstrow(variables)` writes the variable names in the first row, and `replace` overwrites an earlier version of the file. Before exporting, check the number of households behind each figure, for example with `tabulate age_group if implicate == 1`.

Good practice

Split a project into short do-files with one task each (build the data, compute the statistics, export the tables) and a *master* do-file that runs them in order with `do 01_build.do`, `do 02_stats.do`, and so on. Anyone can then reproduce every result by running one file.

3.10 Summary

Command	Use with survey microdata
`append`	Stack the implicate files into one dataset.
`label define/values`	Attach readable texts to numeric codes.
`tabulate, generate()`	Create one dummy per category (shares).
`summarize, detail`	Weighted mean and percentiles, stored in `r()`.
`quietly`	Run a command without printing its output.
`scalar`	Store one number for later use.
`table`	Display statistics by implicate or group.
`preserve/restore`	Protect the microdata in memory.
`collapse`	Turn microdata into a dataset of statistics.
`expand`	Duplicate rows (implicate 0 for `mi`).
`mi import/svyset/estimate`	Point estimates and standard errors with five implicates.
`export excel`	Take aggregated results out.

References

Antón, J.-I. *Introducción al análisis de datos con el software Stata*. Course materials, Universidad de Salamanca. https://jianton.github.io/curso_stata.html

Banco de España. *Encuesta Financiera de las Familias: Libro de Código*. Madrid: Banco de España.

Household Finance and Consumption Network (2026). *The Household Finance and Consumption Survey: Methodological report for the 2023 wave*. ECB Statistics Paper Series No 52.

Long, J. S. (2009). *The Workflow of Data Analysis Using Stata*. College Station, TX: Stata Press.

StataCorp. *Economics: Features for economists*. College Station, TX: StataCorp LLC. https://www.stata.com/disciplines/economics/

StataCorp. *Stata Reference Manuals*: `append`, `collapse`, `egen`, `forvalues`, `mi estimate`. College Station, TX: Stata Press. https://www.stata.com/manuals/

UCLA Office of Advanced Research Computing. *Stata resources and learning modules*. https://stats.oarc.ucla.edu/stata/